

Name: Hatem Ibrahim  
Student ID: 40265641

## **Task 1: Repository Setup and GitHub Workflow**

### **Purpose of AI Use**

Project organization and Git/GitHub guidance. AI was used to understand folders, branches, .gitkeep files, .gitignore, issue templates, pull-request templates, and repository protection options.

### **Chat Prompt/Response**

#### **Prompt 1:**

Can you explain a simple way to organize our existing GitHub repository into frontend, backend, database, documentation, meeting-minutes, sprint-deliverable, and individual AI-log areas

#### **Prompt 2:**

Can you guide me through the terminal commands for creating and tracking folders, using .gitkeep, and checking Git status. Please keep the workflow appropriate for a university project and explain why each step is needed.

### **AI-Suggested Content**

AI suggested a repository layout separating frontend, backend, Supabase database files, documentation, meeting minutes, sprint deliverables, and individual AI logs. It also explained terminal commands for creating folders, tracking empty directories, checking Git status, and committing specific files.

### **Reflection**

AI was useful for explaining why Git does not track empty folders and why generated dependencies and secrets should not be committed. Commands were run individually and their output was checked before continuing.

### **Responsible Person**

Hatem Ibrahim

## **Task 2: Profile Management Database Design**

### **Purpose of AI Use**

System design, database modeling, and technical explanation for the profile management feature.

### **Chat Prompt/Response**

**Prompt 1:**

Can you review the existing repository files and explain how the login database, profile backend, and profile frontend relate before I make database changes

**Prompt 2:**

Can you guide me through designing a profile management migration that matches the existing login database, and explain the tables, relationships, constraints, and security policies I need to understand

**AI-Suggested Content**

AI reviewed the existing repository context and proposed a schema that links public profiles to Supabase Auth users by UUID. It recommended profile fields for the UI and separate one-to-many tables for work experience, education, and skills, together with foreign keys, indexes, timestamps, triggers, and Row Level Security.

**Reflection**

The interaction helped explain why the profile UUID should match auth.users.id and why repeating profile sections belong in related tables rather than additional columns in profiles.

**Responsible Person**

Hatem Ibrahim

**Task 3: Migration Error Diagnosis and Repair****Purpose of AI Use**

Debugging and database migration review after the local Supabase reset reported a bigint to UUID cast error.

**Chat Prompt/Response****Prompt 1:**

Supabase reports cannot cast type bigint to uuid at ALTER COLUMN id TYPE uuid USING id::uuid. Can you explain why this happens and help me find out which migration is causing it

**Prompt 2:**

Can you explain the safe options for correcting the earlier migration so a clean local database reset works without creating problems for the database history

**AI-Suggested Content**

AI identified that PostgreSQL cannot directly cast a bigint identity value to uuid using id::uuid. It explained that the failure came from an earlier migration and proposed a deterministic

hexadecimal conversion so the full migration history could be replayed locally. It also explained the difference between repairing migration history for local rebuilds and changing an already-applied remote database.

### **Reflection**

AI helped locate the cause in the earlier migration instead of incorrectly changing the new profile schema. The main lesson was to test the entire migration chain from an empty local database.

### **Responsible Person**

Hatem Ibrahim

## **Task 4: Supabase CLI and Docker Troubleshooting**

### **Purpose of AI Use**

Development-environment setup and troubleshooting on macOS, including missing com executable permissions.

### **Chat Prompt/Response**

#### **Prompt 1:**

I installed Docker Desktop. Can you guide me step by step through checking Docker, starting the local Supabase services, and preparing to test the migrations?

#### **Prompt 2:**

I receive 'node\_modules/.bin/supabase: Permission denied' on macOS. What does this error mean, and how should I correct the executable permission before continuing?

### **AI-Suggested Content**

AI explained the roles of Docker Desktop and the Supabase CLI, recommended using the repository's local CLI with npx, and provided step-by-step terminal checks. When the local executable returned Permission denied, AI identified the file-permission cause and recommended restoring executable permission before rerunning the CLI.

### **Reflection**

This assistance was useful for understanding the difference between installing a dependency, running it through npx, and starting its Docker-based local services.

### **Responsible Person**

Hatem Ibrahim

## Task 5: Database Testing and Integration Review

### Purpose of AI Use

Test planning, manual verification, and review of whether the frontend, backend, and database could operate together.

### Chat Link or Prompt/Response

#### Prompt 1:

Can you help me create a clear local test plan for the migrations, profile tables, authentication/profile trigger, foreign keys,

and security rules before I push my branch?

#### Prompt 2:

Can you review the current frontend, backend, and database and explain what can be tested now and what integration

work is still missing for a complete end-to-end website test?

### AI-Suggested Content

AI proposed rebuilding the local database with db reset, checking tables and columns in Supabase Studio, creating a test

authentication user, confirming automatic profile creation, and testing related profile records and foreign-key protection. It also

reviewed the repository and identified that the frontend and backend still required UUID and API integration before a complete

end-to-end website test would be possible.

### Validation

The test plan was compared with the schema's triggers, constraints, and RLS design. The static profile interface was previewed

separately so UI behavior was not confused with database integration. The limitations were documented instead of claiming an

unverified full-system test.

## Decision

Accepted as a testing plan and integration assessment.

## Reflection

AI helped distinguish migration testing, database behavior testing, backend unit testing, UI previewing, and true end-to-end testing.

This made the validation claims more accurate.

## Responsible Person

Hatem Ibrahim